

SAE S1.01

Projet "chatbot"

Farogh Hedari et Camille Goyet

Introduction

The goal of this SAE project is to develop a chatbot. Chatbots are computer programs that simulate human conversations (written or spoken), allowing users to interact with a device as if they were interacting with a person. For this project, the chatbot generates written general-knowledge responses to user inquiries by choosing among pre-recorded responses.

The chatbot's data used to create its responses is stored in text files (one for answers, one for questions and answers, and one for potential typing errors). All interactions with this chatbot occur through dialogue. When a user inputs a question, the chatbot searches its database (in this case, a text file) for the appropriate answer as efficiently and reliably as possible. To accomplish this task, the chatbot divides questions into two sections: one that contains "tool" words and the other that describes the question's "theme." The chatbot then uses data from its databases to construct a context to answer the user's question and determine which model would yield the best results.

A chatbot provides users with an opportunity to communicate with it through chat messages. Users may ask questions and receive answers if the chatbot knows the answer, but if not, users can provide information to help the chatbot learn. Users can teach the chatbot anything, including incorrect information; therefore, users must ensure that the answers they provide do not corrupt the question-and-answer database or file. We found that the chatbot learns from what users told it and will include that information in the database of what it knows at the end of the answer file. Because of this, the answer database will be bigger, which means that the chatbot can answer more questions.

The chatbot also can understand questions that are composed exclusively of tool words using the information provided by the user in the previous question. For example, once the chatbot answered "In what year was the invention of the

stethoscope?" it can then answer follow-up questions such as "by who?" or "where?". Thus, users won't have to repeat the exact same phrases for the same type of context when making the same inquiry a second time.

The chatbot utilizes different ways users can enter questions using synonymous and various words. In order to do this, the chatbot comes with a Thesaurus.txt file to accompany the question and answer files when initializing or starting the chatbot. The Thesaurus.txt file consists of Words and synonyms/variants of those words. It converts the words into Canonical Form, an individual form, and thus makes it possible to connect all related words to Canonical Form.

Organisation

Through the use of the chatbot, we were able to allocate tasks by organizing our project into stages. Essentially, we allocated time at the beginning of the project to thoroughly understand the project's objectives as per SAE. Once this was completed, we generated Sequential drawings detailing the process of construction. After analyzing all aspects of the code and also the mini-responses file, we collectively developed the way to build the program.

In the next phase of implementation, we began the Index class' methods to create both input and output vectors, as well as add and search for inputs/outputs. Once we finished creating input/output methods for Index, we created a list of candidate responses for the project through the Utilitaire class' utility functions. Once we had all these candidate responses, we integrated them with the Chatbot class and verified that our methods worked properly together. By running the program at this point, we could see the vector list—what we identified as the response identifiers, context words, and tool words. We then moved on to developing our coherent methods for selecting responses from the list of candidates. This marked the end of Part 1 of the project and was the longest section because of numerous

inconsistencies across the methods, particularly in the way the methods displayed results. The major attempt to track down the source of the issues came from the developers being confused as to why the program would continuously respond, "I don't know." After hours of investigation, we found that the problem existed in the Utilitaire class, specifically, within the response construction and response selection methods—both of which had contributed values to the response identifier list.

Although developing our own chatbot in Part 2 included improving upon everything from the word variants (synonyms) used within the chatbot, we found the creation of the text file and linking it to the methods very simple. The contextualisation step was equally simple because there was only one question with a tool word (the last) to contextualise.

The next step demonstrated that the program cannot distinguish between the use of the word 'en' in relation to dates or places. Thus, the program will return either of the two. To resolve this issue, all we had to do was implement a method to check whether or not a string contains an integer, before updating the methods accordingly. Initially, the function of the tool word 'NUM' was difficult to understand, however over time we came to understand how using NUM allows us to differentiate between dates and locations. As a result, our program can now return the correct information to users.

Finally we have added the knowledge addition feature. In order to accomplish this, we needed a new user message: "I am going to teach you." Using that new user message, we were able to finish out the methods needed for the addition of indexes. However, the adding of indexes has been done only on the surface level. The user could add information only while the program was running. Once the program was stopped and then restarted, all the questions that the user had taught the chatbot would be lost.

To maintain the user's input answers, we created methods in the chatbot class to allow the user's input answers to be saved. Updates to the indexes and the individual user files will be performed based on the new knowledge that was added by the user. The final benefit of preserving user input knowledge is that knowledge will persist over time to preserve the new information.

In closing, during our testing, we improved our database. Although we completed the project, the limitations of the chatbot still exist, which is true for most chatbots.

Mathematical Analysis

The project uses an index structure to optimize search complexity.

We compare the naive approach (without index) with our index-based approach.

Parameters:

- $nr = 100,000$ (number of responses in database)
- $nmv = 10,000$ (number of distinct non-tool words in all responses)
- $nmr = 5$ (average non-tool words per response)
- $nmq = 3$ (non-tool words in question)
- $nrpm = nr * nmr / nmv = 50$ (responses per word, assuming balanced distribution)

Naïve Approach (without Index):

For each response, we must compare every word in the response with every word in the question.

$$\begin{aligned}\text{Complexity} &= nr * nmr * nmq \\ &= 100,000 * 5 * 3 \\ &= 1,500,000 \text{ comparisons}\end{aligned}$$

Index Approach (with Index):

For each word in the question, we look up the index to find candidate responses.

$$\begin{aligned}\text{Complexity} &= nmq * nrpm \\ &= 3 * 50 \\ &= 150 \text{ comparisons}\end{aligned}$$

$$\text{Speedup Factor} = 1,500,000 / 150 = 10,000x$$

This demonstrates the significant performance improvement achieved by using an index structure. The index reduces complexity from $O(nr * nmr * nmq)$ to $O(nmq * nrpm)$, providing a 10,000x speedup for our test parameters.